

MediCare: Hospital Information and Management Database — Project Report

Team 1 · Sep 30, 2026 ·

1. Problem Understanding and Requirements

MediCare replaces scattered hospital records with one connected, 11-table MySQL database that links patients, doctors, appointments, admissions, prescriptions and payments.

1.1 The problem

Hospitals produce a constant stream of interrelated data: registrations, doctor schedules, room assignments, prescriptions and bills. In small and mid-sized facilities this data often sits in paper files, logbooks and separate spreadsheets kept by different desks. The consequences are predictable: duplicate patient profiles, double-booked rooms, misplaced medical history, and bills that do not match the care actually given.

MediCare fixes this by keeping every record in one relational database, so each desk works from the same up-to-date information and every clinical or financial record can be traced back to the right patient and visit.

1.2 Objectives

- **One central hub:** patients, doctors, appointments, rooms, prescriptions and billing in one connected system.
- **Cleaner records:** fewer duplicate profiles and fewer data-entry errors, enforced through keys and constraints.
- **Connected history:** diagnoses, prescriptions and payments tied to the correct patient and visit.
- **Practical reporting:** SQL queries for revenue, doctor workload and room usage.
- **Flexible base:** a simple, normalised structure that later modules (insurance, pharmacy stock) can extend.

1.3 Functional requirements

Area	What the system must do	Tables involved
Patient registration	Store name, date of birth, gender, phone and blood group; give every patient one unique ID; keep the full visit history	patients
Doctors and departments	Organise doctors by department, specialisation and consultation fee	departments, doctors
Appointments	Book a patient with a doctor for	appointments

Area	What the system must do	Tables involved
	a date and time; track Scheduled, Completed or Cancelled	
Clinical records	Record diagnoses and prescriptions that result from an appointment	diagnoses, prescriptions
Pharmacy	Keep a medicine master list; allow many medicines per prescription with dosage and duration	medicines, prescription_medicines
Admissions	Track room inventory, daily rates, availability, admission and discharge	rooms, admissions
Billing	Record payments against an appointment or an admission with amount, date and mode	payments
Reporting	Answer revenue, workload and room-usage questions in SQL	all

1.4 Business rules and how the database enforces them

Every business rule below is traced to the mechanism in `create_tables.sql` that supports it, and one is honestly marked as not enforced by the database.

#	Business rule	Enforcement in the schema
1	Every patient has one unique ID	patient_id is an auto-increment primary key
2	Each doctor belongs to exactly one department	doctors.department_id is NOT NULL with a foreign key
3	Each appointment has one patient and one doctor	Both foreign keys in appointments are NOT NULL
4	A room hosts one active admission at a time	Not enforced by the database: MySQL CHECK cannot compare rows, so this is an application-level rule (see 4.6)
5	A prescription holds many medicines; a medicine appears	Bridge table prescription_medicines with

#	Business rule	Enforcement in the schema
	in many prescriptions	a composite primary key
6	Diagnoses and prescriptions exist only against an appointment	appointment_id is NOT NULL with a foreign key in both tables
7	Every payment belongs to a patient and traces to an appointment or admission	patient_id NOT NULL plus chk_payment_reference
8	Appointment status is one of three values	chk_appointment_status
9	Prescribed medicines come from the master catalogue	Foreign key fk_pm_medicine

1.5 Assumptions

1. A payment covers one appointment or one admission, not a combined bill.
2. Discharge cannot precede admission (enforced by chk_admission_dates).
3. A patient has at most one active admission at a time (assumed, not enforced).
4. All amounts are in a single currency.
5. A doctor's consultation fee is fixed and stored once on the doctor's row.
6. Historical records (visits, admissions, payments) are permanent, so foreign keys on them use RESTRICT.

1.6 Changes from the suggested design

The project brief allows changes when they are explained. The team made these:

- **payments gained appointment_id and admission_id**, both nullable, with a CHECK that at least one is filled. The suggested table had no way to say what a payment was for.
- **Payment modes** in the schema are Cash, Card, UPI and Online. The early requirements draft also listed Insurance; insurance claims remain out of scope.
- **patients has no address column**. The final table follows the brief's attribute list.

1.7 Scope

In scope: registration, doctor and department mapping, appointment tracking, diagnoses, admissions and discharge, a medicine catalogue with prescriptions, payment logging and SQL reporting. Left for future work: patient portal and online booking, duty rosters, bed-level tracking, live pharmacy stock, invoice printing, insurance claims, dashboards, and role-based access control.

2. ER Diagram and Relational Design

The design has 11 entities connected by 1:N relationships, with one many-to-many relationship (prescriptions to medicines) resolved through a bridge table.

2.1 Entity-relationship diagram

The diagram shows every entity with its attributes, the relationships between them and their cardinalities. Primary keys are underlined and foreign keys are marked FK.

Figure 1. The team's ER diagram (Chen notation), saved as `diagrams/er_diagram.png`.

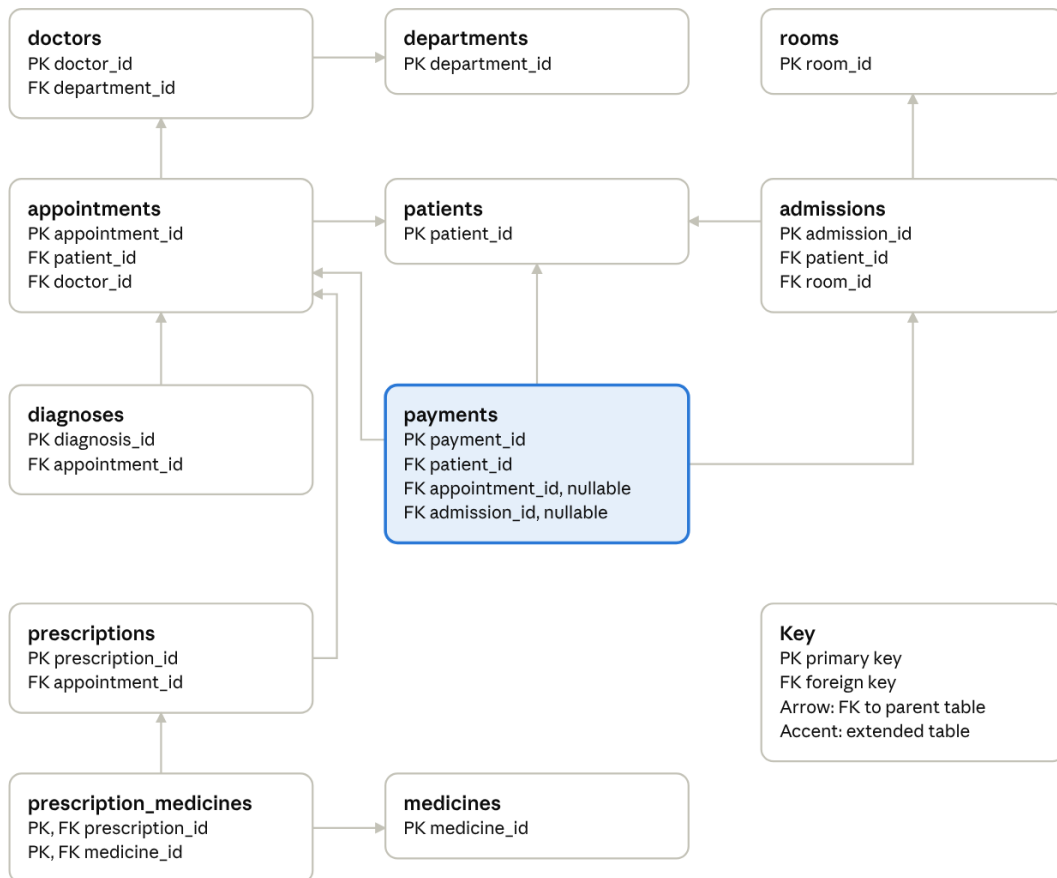
Relationship	Entities	Cardinality	Implemented by
Employs	departments to doctors	1 : N	doctors.department_id
Conducts	doctors to appointments	1 : N	appointments.doctor_id
Books	patients to appointments	1 : N	appointments.patient_id
Results in	appointments to diagnoses	1 : N	diagnoses.appointment_id
Generates	appointments to prescriptions	1 : N	prescriptions.appointment_id
Includes	prescriptions to prescription_medicines	1 : N	prescription_medicines.prescription_id
Contains	medicines to prescription_medicines	1 : N	prescription_medicines.medicine_id
Admitted	patients to admissions	1 : N	admissions.patient_id
Assigned to	rooms to admissions	1 : N (one active at a time)	admissions.room_id
Settled by	appointments to payments	1 : N	payments.appointment_id (nullable)
Settled by	admissions to payments	1 : N	payments.admission_id (nullable)
Pays	patients to payments	1 : N	payments.patient_id

Where a child's foreign key is NOT NULL, the child entity participates totally in the relationship: a doctor must have a department, an appointment must have a patient and a doctor, and a diagnosis or prescription must have an appointment. The two payment foreign keys are nullable, so payments participate partially in each and the CHECK constraint requires at least one of them.

2.2 Relational schema

The ER model maps to the 11 tables below. Underlined keys in the diagram become the primary keys here; every relationship becomes a foreign key on the "many" side.

Every relationship is a foreign key pointing to its parent table



relational schema · 11 tables, keys only

The arrows run from each foreign key to the table it references, so the parent (the "one" side) is always at the arrowhead. **payments** is highlighted because it is the one table extended beyond the brief.

Table	Primary key	Foreign keys	Other attributes
departments	<u>department_id</u>	none	name (unique), location
doctors	<u>doctor_id</u>	department_id to departments	name, specialisation, consultation_fee
patients	<u>patient_id</u>	none	name, dob, gender,

Table	Primary key	Foreign keys	Other attributes
			phone, blood_group
rooms	room_id	none	room_type, daily_charge, is_available
appointments	appointment_id	patient_id, doctor_id	appointment_date, status
diagnoses	diagnosis_id	appointment_id	description, diagnosis_date
admissions	admission_id	patient_id, room_id	admit_date, discharge_date (NULL = still admitted)
medicines	medicine_id	none	name, manufacturer, unit_price
prescriptions	prescription_id	appointment_id	issued_date
prescription_medicines	(prescription_id, medicine_id)	both parts are foreign keys	dosage, duration_days
payments	payment_id	patient_id, appointment_id (nullable), admission_id (nullable)	amount, payment_date, mode

2.3 Key design decisions

Payments link to an appointment or an admission. The suggested payments table said who paid and how much, but not what for. The team added two nullable foreign keys and a CHECK that at least one is filled, so every payment traces back to a visit or a stay. The sample data fills exactly one of the two on every payment.

A bridge table resolves prescriptions to medicines. A prescription can hold many medicines and a medicine can appear on many prescriptions, so neither side can carry a foreign key alone. prescription_medicines has a composite primary key and carries dosage and duration_days, which describe one medicine on one prescription and belong on neither parent table.

Referential actions follow the type of data.

Data type	Foreign keys	ON DELETE	Reason
Master and reference data	doctors, patients, rooms, departments, medicines referenced	RESTRICT	A doctor, patient, room, department or medicine cannot be

Data type	Foreign keys	ON DELETE	Reason
	by other tables		deleted while records still refer to it
Dependent detail	diagnoses, prescriptions, prescription_medicines to their parents	CASCADE	These records have no meaning without their appointment or prescription
Financial history	payments.appointment_id, payments.admission_id, payments.patient_id	RESTRICT	Payments are permanent history and are never deleted automatically

All foreign keys use ON UPDATE CASCADE except the two on payments that point to appointments and admissions. Those use ON UPDATE RESTRICT on purpose, so a key renumbering can never silently ripple into billing records.

3. Normalisation and Database Structure

All 11 relations are in Third Normal Form (3NF): every non-key attribute depends on the key, the whole key, and nothing but the key.

3.1 Functional dependencies

In MediCare the primary key of each relation determines its other attributes. Foreign keys describe relationships but do not make the parent's attributes depend on the child: `doctor_id` in `appointments` identifies the doctor, while the doctor's name, specialisation and fee stay in `doctors`.

Relation	Functional dependency	Highest normal form
departments	department_id → name, location	3NF
doctors	doctor_id → name, specialisation, department_id, consultation_fee	3NF
patients	patient_id → name, dob, gender, phone, blood_group	3NF
appointments	appointment_id → patient_id, doctor_id, appointment_date, status	3NF
rooms	room_id → room_type, daily_charge, is_available	3NF

Relation	Functional dependency	Highest normal form
admissions	admission_id → patient_id, room_id, admit_date, discharge_date	3NF
diagnoses	diagnosis_id → appointment_id, description, diagnosis_date	3NF
prescriptions	prescription_id → appointment_id, issued_date	3NF
medicines	medicine_id → name, manufacturer, unit_price	3NF
prescription_medicines	(prescription_id, medicine_id) → dosage, duration_days	3NF
payments	payment_id → patient_id, appointment_id, admission_id, amount, payment_date, mode	3NF

3.2 From one large record to 11 relations

Unnormalised form (UNF). A hospital record could be imagined as one wide relation holding everything at once:

```
HOSPITAL_RECORD(patient_id, patient_name, dob, gender, phone, blood_group,
  doctor_id, doctor_name, specialisation, department_id, department_name,
  department_location, appointment_id, appointment_date, status,
  diagnosis_id, description, diagnosis_date, prescription_id, issued_date,
  medicine_id, medicine_name, manufacturer, unit_price, dosage, duration_days,
  admission_id, room_id, room_type, daily_charge, admit_date, discharge_date,
  payment_id, amount, payment_date, mode)
```

This repeats department, doctor, medicine, room and patient details on every transaction, and a prescription with several medicines forms a repeating group.

First Normal Form. Every attribute holds one atomic value and repeating groups become their own rows:

Problem in the unnormalised record	Fix
Several medicines in one prescription	One row per prescription and medicine pair in prescription_medicines
Several appointments for one patient	One row per visit in appointments
Several admissions for one patient	One row per stay in admissions

Problem in the unnormalised record	Fix
Several diagnoses for appointments	One row per diagnosis in <code>diagnoses</code>

Second Normal Form. Ten relations have a single-column primary key, so a partial dependency is impossible. The only composite key is (`prescription_id`, `medicine_id`) in `prescription_medicines`. Its attributes `dosage` and `duration_days` describe one medicine on one prescription, so they depend on the complete key. Neither `prescription_id` $\rightarrow$ `dosage` nor `medicine_id` $\rightarrow$ `dosage` holds, because the same medicine is dosed differently on different prescriptions.

Third Normal Form. No non-key attribute depends on another non-key attribute. The clearest case is doctors and departments:

	Design	Problem or result
Before	<code>doctors(doctor_id, name, department_id, department_name, department_location)</code>	<code>doctor_id</code> $\rightarrow$ <code>department_id</code> $\rightarrow$ <code>department_name</code> , <code>department_location</code> is a transitive dependency
After	<code>doctors(doctor_id, name, specialisation, department_id, consultation_fee)</code> and <code>departments(department_id, name, location)</code>	Department details are stored once and <code>department_id</code> is a foreign key

The same principle keeps appointments free of doctor details, admissions free of room type and daily charge, and payments free of patient, appointment or admission details.

3.3 Anomalies avoided

Anomaly	How the design avoids it
Update	Department, doctor, room and medicine details live in their own tables, so one change does not touch many transaction rows
Insert	A department, doctor, medicine or room can be added without an unrelated appointment or payment existing first
Delete	Deleting an appointment (which cascades to its diagnoses and prescriptions) never removes a doctor, patient, medicine or department

3.4 Integrity constraints

The schema applies all five constraint types the brief asks for, plus referential actions (Section 2.3).

Constraint	Where it is used
PRIMARY KEY	One auto-increment key per table; composite key on <code>prescription_medicines</code>
FOREIGN KEY	12 foreign keys linking child tables to their parents
NOT NULL	Identity and business-critical fields, including every mandatory foreign key
UNIQUE	<code>departments.name</code>
DEFAULT	<code>appointments.status = 'Scheduled';</code> <code>rooms.is_available = TRUE</code>
CHECK	Fees, charges and prices ≥ 0 ; payment amount > 0 ; <code>duration_days > 0</code> ; gender in Male, Female, Other; <code>blood_group</code> in the eight ABO/Rh types; appointment status in Scheduled, Completed, Cancelled; payment mode in Cash, Card, UPI, Online; <code>discharge_date</code> NULL or on/after <code>admit_date</code> ; <code>chk_payment_reference</code>

Fixed-value fields use CHECK (... IN (...)) rather than MySQL ENUM, because CHECK ports to PostgreSQL or SQL Server while ENUM is MySQL-specific.

3.5 Indexes

MySQL indexes every foreign key column automatically. Five explicit indexes cover the other columns the queries filter or group on:

Index	Column(s)	Supports
<code>idx_appointments_date</code>	<code>appointment_date</code>	Date filtering on appointments
<code>idx_appointments_status</code>	<code>status</code>	Completed and cancelled counts (Q10, Q11)
<code>idx_admissions_dates</code>	<code>admit_date</code> , <code>discharge_date</code>	Length-of-stay calculation (Q9)
<code>idx_payments_date</code>	<code>payment_date</code>	Monthly revenue (Q7)
<code>idx_patients_name</code>	<code>name</code>	Looking up a patient before registering a duplicate

4. Database Implementation and Data Quality

The schema is implemented in MySQL 8.0.16 or later and loaded with 228 rows of deliberately designed sample data across all 11 tables, so every business query returns a real result.

4.1 Environment and run order

MySQL 8.0.16 is the minimum version, because earlier versions accept CHECK constraints but do not enforce them. That would let invalid rows through silently, for example a negative payment or a discharge before admission. The three scripts run in this order and each depends on the one before:

```
mysql -u your_username -p < schema/create_tables.sql
mysql -u your_username -p < data/insert_data.sql
mysql -u your_username -p < queries/queries.sql
```

create_tables.sql begins with DROP DATABASE IF EXISTS medicare, so it deletes any existing medicare database. This is what makes the script safe to re-run from a clean state.

4.2 Implementation decisions

Decision	Choice	Reason
Re-run safety	DROP DATABASE IF EXISTS, then CREATE DATABASE, then defensive DROP TABLE IF EXISTS in reverse dependency order	Rebuilds from a clean state at any time, even when only the tables need rebuilding
Creation order	departments, doctors, patients, rooms, appointments, diagnoses, admissions, medicines, prescriptions, prescription_medicines, payments	Parents are created before children, so every foreign key target already exists
Money fields	DECIMAL(10,2) for fees, charges, prices and amounts	Avoids the rounding errors of FLOAT on currency
Dates	DATETIME for appointment_date; DATE for admit, discharge, issue and payment dates	Time of day matters for scheduling; day-level detail is enough elsewhere
Fixed-value fields	CHECK (... IN (...)) instead of ENUM	Portable to PostgreSQL and SQL Server
Storage engine	InnoDB on every table	Required for foreign keys and transactions

The constraint block of payments shows the billing design in three lines:

```
CONSTRAINT chk_payment_amount    CHECK (amount > 0),
CONSTRAINT chk_payment_mode      CHECK (mode IN ('Cash', 'Card', 'UPI', 'Online')),
CONSTRAINT chk_payment_reference CHECK (appointment_id IS NOT NULL OR admission_id IS NOT NULL)
```

4.3 Sample data design

The data in `insert_data.sql` was built on purpose rather than generated at random. Each choice below exists so that a specific query returns a meaningful, non-tied answer.

Design choice	Why it matters
4 of 18 patients have no appointments	Q4 returns real rows
Two patients (Arjun Reddy, Amit Bansal) each have 2 admissions	Q6 (<code>HAVING COUNT > 1</code>) has results
Appointments and payments span January to June 2026	Q7 groups into six distinct months
Ibuprofen and Amlodipine are prescribed far more than the rest	Q5 has a clear top result (9 and 8 prescriptions)
Consultation fees range from ₹550 to ₹1500 across departments	Q3 shows a real spread, from ₹575 to ₹1350 on average
29 appointments Completed, 2 Scheduled, 1 Cancelled	Exercises the <code>status CHECK</code> and feeds Q11
Two admissions have <code>discharge_date</code> NULL	Represents patients currently admitted; Q9 correctly leaves them out of the average stay

4.4 Row counts after loading

Table	Rows
departments	5
doctors	9
patients	18
rooms	8
medicines	12
appointments	32
admissions	10
diagnoses	29
prescriptions	29
prescription_medicines	39
payments	37

Table	Rows
Total	228

4.5 Data quality

The sample data is consistent with the schema and with the business questions:

- **Constraints hold.** Every INSERT respects the primary keys, foreign keys and CHECK constraints, and the three scripts run end to end on a clean database without errors.
- **Coverage.** All 11 tables are populated, and all 11 queries return non-empty, non-trivial results.
- **Billing coverage.** All 29 completed appointments have exactly one payment, and 8 admissions have a linked payment. The 3 cancelled or scheduled appointments have none. Every payment fills exactly one of appointment_id and admission_id.
- **Realistic variety.** The data mixes all four payment modes, all eight blood groups, all three appointment statuses and four room types (General, Semi-Private, Private, ICU).
- **Identifiers.** Each patient has a distinct 10-digit phone number in the sample.

4.6 Limitations and future work

- **Overlapping admissions are not blocked.** "One active admission per room" and "one active admission per patient" are business rules the database cannot enforce with a CHECK, which cannot compare rows. They are handled at the application level, and the team lists this as a known simplification.
- **Fee history is not kept.** A doctor's consultation fee is stored once, so a fee change is not reflected historically and past visits do not retain the fee that applied.
- **One payment per transaction.** Partial and instalment payments are out of scope.
- **Next steps.** Insurance claims, live pharmacy stock, bed-level tracking, duty rosters and role-based access control fit on top of the current tables without restructuring them.

5. SQL Queries and Analysis

Eleven queries answer the seven questions in the brief and four more the team chose; every one returns a real result against the sample data.

All queries are in queries/queries.sql. The results below are from the sample data loaded in Section 4. Q1 to Q7 are the required questions and Q8 to Q11 are the team's own.

Query	Business question	SQL concepts	Headline result
Q1	Which doctors treated the most patients?	JOIN, COUNT(DISTINCT), GROUP BY	Dr. Suresh Iyer, 4 patients
Q2	Which department has the most appointments?	Multi-table JOIN, LIMIT	Cardiology, 9

Query	Business question	SQL concepts	Headline result
Q3	Average consultation fee by department	AVG, GROUP BY	₹575 to ₹1350
Q4	Registered patients who never visited	LEFT JOIN ... IS NULL	4 patients
Q5	Most frequently prescribed medicines	Bridge-table JOIN, COUNT	Ibuprofen, 9
Q6	Patients admitted more than once	GROUP BY, HAVING	2 patients
Q7	Monthly revenue	DATE_FORMAT, SUM	₹95,650 over 6 months
Q8	Room occupancy	LEFT JOIN, COUNT	Room 4 never used
Q9	Average length of stay by room type	DATEDIFF, AVG, WHERE	ICU, 3.5 days
Q10	Consultation revenue per doctor	Conditional JOIN, arithmetic	Dr. Rohan Mehta, ₹6,000
Q11	Cancellation rate per department	SUM(CASE WHEN ...)	Dermatology, 33.3%

Q1. Doctors who treated the most patients

The query counts distinct patients per doctor rather than appointments, so a doctor who sees one patient five times does not outrank one who sees five different patients.

```
SELECT d.doctor_id, d.name AS doctor_name, dept.name AS department,
       COUNT(DISTINCT a.patient_id) AS patients_treated
FROM doctors d
JOIN departments dept ON dept.department_id = d.department_id
JOIN appointments a ON a.doctor_id = d.doctor_id
GROUP BY d.doctor_id, d.name, dept.name
ORDER BY patients_treated DESC;
```

Result: Dr. Suresh Iyer (Orthopedics) leads with 4 distinct patients, followed by Dr. Ananya Kulkarni (Pediatrics) with 3. Five doctors have treated 2 patients and two have treated 1. The count includes every appointment status.

Q2. Busiest department

```
SELECT dept.department_id, dept.name AS department,
       COUNT(a.appointment_id) AS total_appointments
FROM departments dept
JOIN doctors d ON d.department_id = dept.department_id
JOIN appointments a ON a.doctor_id = d.doctor_id
GROUP BY dept.department_id, dept.name
```

```
ORDER BY total_appointments DESC
LIMIT 1;
```

Result: Cardiology has the most appointments, 9 of the 32 in the database. It joins three tables (appointments, doctors, departments) because appointments store only a doctor, not a department.

Q3. Average consultation fee by department

The query returns COUNT(d.doctor_id) beside the average, so a department's average is read alongside the number of doctors behind it.

```
SELECT dept.name AS department, COUNT(d.doctor_id) AS num_doctors,
       ROUND(AVG(d.consultation_fee), 2) AS avg_consultation_fee
FROM departments dept
JOIN doctors d ON d.department_id = dept.department_id
GROUP BY dept.department_id, dept.name
ORDER BY avg_consultation_fee DESC;
```

Department	Doctors	Average fee (₹)
Cardiology	2	1350.00
Dermatology	1	1000.00
Orthopedics	2	850.00
Pediatrics	2	675.00
General Medicine	2	575.00

Q4. Patients who never visited

A LEFT JOIN with IS NULL returns the patient details directly, which a NOT EXISTS form would also do but with a subquery.

```
SELECT p.patient_id, p.name, p.phone
FROM patients p
LEFT JOIN appointments a ON a.patient_id = p.patient_id
WHERE a.appointment_id IS NULL
ORDER BY p.patient_id;
```

Result: Four registered patients have no appointment: Ramesh Chandran, Fatima Ansari, Sanjay Gowda and Rekha Iyengar (patient IDs 15 to 18). These are the records a front desk could follow up on.

Q5. Most frequently prescribed medicines

Medicine frequency cannot be read from prescriptions or medicines alone, so the query joins through the prescription_medicines bridge table. This is the query that most directly shows why the many-to-many relationship needed its own table.

```
SELECT m.medicine_id, m.name AS medicine_name,
       COUNT(pm.prescription_id) AS times_prescribed
FROM medicines m
JOIN prescription_medicines pm ON pm.medicine_id = m.medicine_id
GROUP BY m.medicine_id, m.name
ORDER BY times_prescribed DESC;
```

Result: Ibuprofen 400mg is prescribed most often (9 prescriptions), then Amlodipine 5mg (8), Paracetamol 500mg (6) and Cetirizine 10mg (4). Atorvastatin and Metformin appear 3 times each, Insulin Glargine twice, and four medicines once. Medicines that were never prescribed do not appear, because the join is an inner join.

Q6. Patients admitted more than once

The filter applies to an aggregated count, so it needs `HAVING` rather than `WHERE`.

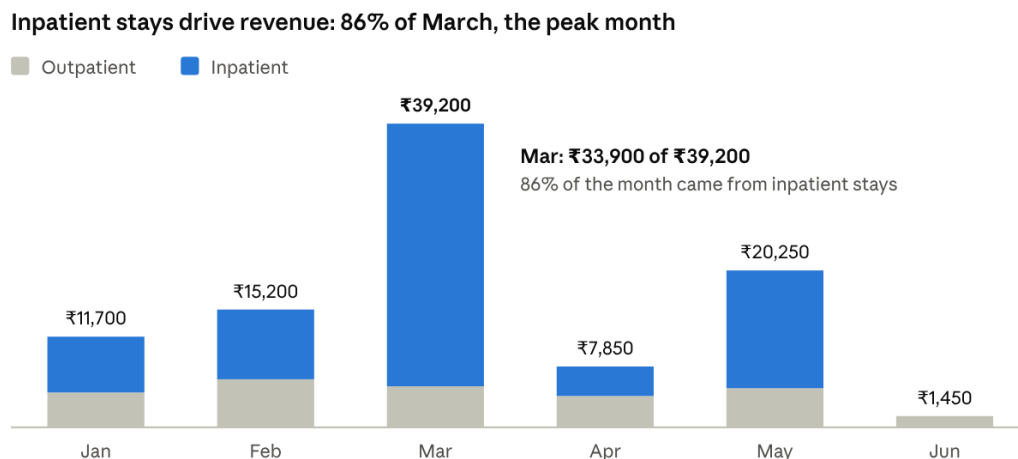
```
SELECT p.patient_id, p.name, COUNT(ad.admission_id) AS total_admissions
FROM patients p
JOIN admissions ad ON ad.patient_id = p.patient_id
GROUP BY p.patient_id, p.name
HAVING COUNT(ad.admission_id) > 1
ORDER BY total_admissions DESC;
```

Result: Arjun Reddy and Amit Bansal have each been admitted twice.

Q7. Monthly revenue

`DATE_FORMAT(payment_date, '%Y-%m')` groups payments into calendar months regardless of the day. This is the MySQL-specific part of the query; PostgreSQL would use `TO_CHAR(payment_date, 'YYYY-MM')`.

```
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS revenue_month,
       COUNT(*) AS num_payments, SUM(amount) AS total_revenue
FROM payments
GROUP BY DATE_FORMAT(payment_date, '%Y-%m')
ORDER BY revenue_month;
```



sample payments, Jan–Jun 2026 · split by appointment or admission link

Result: The hospital collected ₹95,650 across 37 payments from January to June 2026. March is the peak at ₹39,200, and two inpatient payments (₹22,500 and ₹11,400) make up ₹33,900 of it, so admissions drive the swings in monthly revenue. June is low because the sample data ends on 4 June.

Q8. Room occupancy (team question)

A `LEFT JOIN` from `rooms` to `admissions` keeps a never-used room in the results with a count of zero. A plain `JOIN` would hide unused inventory, which is exactly what a hospital most needs to see.

```
SELECT r.room_id, r.room_type, r.is_available, COUNT(ad.admission_id) AS times_used
FROM rooms r
LEFT JOIN admissions ad ON ad.room_id = r.room_id
GROUP BY r.room_id, r.room_type, r.is_available
ORDER BY times_used DESC;
```

Result: Rooms 1 (General), 3 (Semi-Private) and 6 (Private) were each used twice. Rooms 2, 5, 7 and 8 were used once, and room 4 (Semi-Private) was never used.

Q9. Average length of stay by room type (team question)

The query keeps only discharged admissions, because `DATEDIFF` against a `NULL` discharge date returns `NULL` and would distort the average for patients still in a room.

```
SELECT r.room_type,
       ROUND(AVG(DATEDIFF(ad.discharge_date, ad.admit_date)), 1) AS avg_stay_days,
       COUNT(*) AS discharged_admissions
FROM admissions ad
JOIN rooms r ON r.room_id = ad.room_id
WHERE ad.discharge_date IS NOT NULL
GROUP BY r.room_type
ORDER BY avg_stay_days DESC;
```

Room type	Average stay (days)	Discharged admissions
ICU	3.5	2
Private	3.3	3
Semi-Private	3.0	1
General	2.0	2

Q10. Consultation revenue per doctor (team question)

The query multiplies each doctor's fixed fee by their number of `Completed` appointments. This is a simplification, since it assumes every visit was billed at the list price, but it ranks doctors without needing itemised consultation payments.

```
SELECT d.doctor_id, d.name AS doctor_name, d.consultation_fee,
       COUNT(a.appointment_id) AS completed_appointments,
       d.consultation_fee * COUNT(a.appointment_id) AS estimated_revenue
FROM doctors d
JOIN appointments a ON a.doctor_id = d.doctor_id AND a.status = 'Completed'
GROUP BY d.doctor_id, d.name, d.consultation_fee
ORDER BY estimated_revenue DESC;
```

Result: Dr. Rohan Mehta is first with ₹6,000 (5 visits at ₹1,200), then Dr. Suresh Iyer with ₹5,400 (6 visits at ₹900) and Dr. Kavita Rao with ₹4,500 (3 visits at ₹1,500). Dr. Iyer has the most completed appointments (6), but Dr. Mehta's higher fee puts him ahead.

Q11. Cancellation rate per department (team question)

The `SUM(CASE WHEN ...)` pattern counts cancelled and total appointments in a single pass over the same rows, so the two numbers cannot drift apart.

```
SELECT dept.name AS department, COUNT(*) AS total_appointments,
       SUM(CASE WHEN a.status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled,
       ROUND(100.0 * SUM(CASE WHEN a.status = 'Cancelled' THEN 1 ELSE 0 END)
            / COUNT(*), 1) AS cancellation_rate_pct
FROM appointments a
JOIN doctors d      ON d.doctor_id = a.doctor_id
JOIN departments dept ON dept.department_id = d.department_id
GROUP BY dept.department_id, dept.name
ORDER BY cancellation_rate_pct DESC;
```

Department	Total appointments	Cancelled	Rate (%)
Dermatology	3	1	33.3
Cardiology	9	0	0.0
Orthopedics	8	0	0.0
Pediatrics	5	0	0.0
General Medicine	7	0	0.0

What the queries tell the hospital

- **Demand is concentrated.** Cardiology has the most appointments (9 of 32) and the highest average fee (£1,350).
- **Inpatient care drives revenue.** Admission payments account for most of the peak month, and ICU stays are the longest at 3.5 days on average.
- **Some capacity sits idle.** Room 4 was never used, which Q8 surfaces only because it keeps zero-count rows.
- **Follow-up opportunities exist.** Four registered patients have never had an appointment (Q4), and Dermatology's single cancellation gives it the highest cancellation rate (Q11).

6. GitHub Repository and Documentation

The project lives in one GitHub repository organised so that anyone can read the design, rebuild the database and rerun every query.

6.1 Repository structure

The layout follows the structure recommended in the project brief, with one folder per kind of deliverable and a README that ties them together.

File or folder	Contents
README .md	Project description, team members, setup steps, business questions and design notes

File or folder	Contents
schema/create_tables.sql	Creates the database, all 11 tables, constraints and indexes
data/insert_data.sql	Sample data for every table
queries/queries.sql	All eleven business queries, each with a short comment
diagrams/	ER diagram and relational schema diagram (PNG)
docs/	Requirements, normalisation, design rationale, query results and this report

6.2 Documentation set

Each document answers one question, so nothing is written twice.

Document	What it answers
Requirements (Stage 1)	What problem is being solved, and which rules and assumptions apply
Normalisation	Which functional dependencies exist, and why every table is in 3NF
Design rationale (Stage 9)	Why the schema, constraints and referential actions were chosen
Implementation and queries	How the scripts were built, how the sample data was designed, and what each query does
Query results	The output of Q1 to Q11 on the sample data
Project report	The whole project in one document

6.3 Reproducing the project

Install MySQL 8.0.16 or later, then run the three scripts in order (the commands are in Section 4.1). The database is dropped and rebuilt on each run, so the result is the same every time.

6.4 Team and contributions

Name	USN	Contribution
Adib Fareed Hasan	AU25UG-001	Requirement analysis
A Akshay	AU25UG-067	ER diagram and relational design

Name	USN	Contribution
Abhishek Nayak	AU25UG-088	Repository and documentation
Aditya Sivaji Perumalla	AU25UG-002	Normalisation
Aiman Patil	AU25UG-075	Repository and documentation
Ayushman Anand	AU25UG-007	Implementation and queries

7. Conclusion

MediCare turns the scattered records of a hospital into one connected, normalised database that can answer real operational questions. It has 11 tables in 3NF, 12 foreign keys, CHECK, UNIQUE, DEFAULT and NOT NULL constraints, five supporting indexes, 228 rows of purpose-built sample data and 11 SQL queries, from doctor workload to room occupancy and monthly revenue.

The team's most important design choices were tying every payment to an appointment or an admission, resolving prescriptions to medicines with a bridge table, and matching each foreign key's delete behaviour to the type of data it protects. The main known limitation is that overlapping admissions in one room are prevented by application logic rather than by the database. Insurance claims, live pharmacy stock, bed-level tracking and role-based access control are natural next steps that the current structure can absorb without redesign.